

Automatic Circuit Discovery for Explainable Network Intrusion Detection

Adapting ACDC for Mechanistic Interpretability in NIDS

Based on: Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, Adrià Garriga-Alonso

Presented by: **Anbuselvi S**

Problem Framing: Interpretable Intrusion Detection

Deep learning–based Network Intrusion Detection Systems (NIDS) have become central to modern security infrastructures due to their ability to model complex temporal and statistical attack patterns. However, these models operate as opaque computational systems, providing little insight into *how* intrusion decisions are internally computed.

From a security operations perspective, the inability to explain alerts complicates trust, auditing, and forensic analysis. From a development and research perspective, this opacity hinders debugging, robustness analysis, and systematic improvement of NIDS models.

This work investigates whether **Automatic Circuit Discovery (ACDC)** can be used to expose the internal computational mechanisms responsible for intrusion detection decisions.

Why Existing XAI Is Insufficient for NIDS

- Popular explainability methods (SHAP, LIME):
 - attribute importance to input features
 - rely on local correlations
- These methods do **not** explain:
 - how internal representations interact
 - which internal components are causally necessary
- In security-critical systems:
 - correlated features may be inauthentic
 - explanations must reflect true decision mechanisms
- This motivates a shift from feature attribution to **mechanistic interpretability**.

Applying ACDC to NIDS: Core Idea

- ACDC is applied **post-hoc** to a trained NIDS model
- The NIDS model is treated as a computational system, not a black box
- The key objective is to identify:
 - which internal components
 - and which information pathwaysare **causally responsible** for detecting a specific attack
- This is achieved by systematically testing and pruning the model's internal computational graph

Behavior Definition in the Context of NIDS

- ACDC requires a precise definition of **behavior**
- In NIDS, a behavior corresponds to:
 - the detection of a specific attack class
 - under a defined traffic distribution
- Examples of behaviors:
 - DDoS traffic classified as malicious
 - Port scanning patterns flagged as intrusion
- Each behavior is quantified using:
 - model confidence or logit difference
 - between clean (attack) and corrupted (benign) traffic

Model Representation: NIDS as a Computational Graph

- To apply ACDC, the trained NIDS is converted into a **directed computational graph**
- Graph nodes represent:
 - feature embeddings
 - hidden neurons or layers
 - temporal or attention-based components
- Graph edges represent information flow during inference
- The intrusion prediction node serves as the behavioral output
- ACDC operates by selectively removing edges while preserving behavior

Clean vs Corrupted Traffic in ACDC for NIDS

- ACDC relies on comparing clean and corrupted executions
- In the context of NIDS:
 - **Clean traffic:** flows containing a specific attack pattern
 - **Corrupted traffic:** benign traffic or traffic with attack cues removed
- Corruption strategies may include:
 - shuffling temporal order
 - masking protocol-specific features
 - replacing flow statistics with benign distributions
- This setup enables causal testing of internal components

Activation Patching for Intrusion Detection

- Activation patching replaces internal activations during inference
- For a given component:
 - run the model on clean attack traffic
 - replace selected internal activations with those from corrupted traffic
- Measure the change in intrusion prediction
- If the prediction remains unchanged:
 - the patched component is not causally required

Recursive Pruning of the Computational Graph

- ACDC applies activation patching to graph edges
- Edges are tested in reverse topological order
- For each edge:
 - patch the edge using corrupted activations
 - evaluate the effect on attack detection
- If the effect is below a threshold:
 - the edge is permanently removed
- This process yields a minimal, behavior-preserving circuit

Interpreting Extracted Circuits in NIDS

- The extracted circuit represents the minimal internal computation
- It captures:
 - which features interact
 - which temporal dependencies matter
 - which internal representations are essential
- Circuits are:
 - attack-specific
 - model-specific
- This enables mechanistic understanding of intrusion detection

What ACDC-Based Interpretability Enables

- From a security operations perspective:
 - improved alert trust
 - clearer forensic explanations
- From a developer perspective:
 - identification of spurious correlations
 - debugging of failure cases
- From a research perspective:
 - causal analysis of model behavior
 - principled comparison across attack types

Example: Circuit Discovery for DDoS Detection

- Consider a trained NIDS detecting DDoS attacks
- Behavior of interest:
 - high-confidence prediction for volumetric flooding traffic
- ACDC isolates a circuit that:
 - aggregates packet rate features
 - emphasizes short-term temporal bursts
 - suppresses protocol-specific noise
- The extracted circuit explains *how* the model detects DDoS

Limits of Feature Attribution in NIDS

- Feature attribution methods explain:
 - which inputs influence a decision
- They do not explain:
 - how features are internally combined
 - which intermediate representations matter
 - whether correlations are causal
- ACDC instead explains:
 - internal computation paths
 - causal necessity of components

Evaluating ACDC in Network Intrusion Detection

- Evaluation focuses on two core questions:
 - Does the circuit preserve attack detection performance?
 - Is the circuit significantly smaller than the full model?
- Metrics include:
 - prediction divergence (e.g., KL divergence)
 - detection accuracy under ablation
 - circuit sparsity
- Circuits are validated on held-out attack traffic

System-Level Impact of ACDC-Enabled NIDS

- For Security Operations Centers (SOC):
 - transparent alerts
 - reduced false-positive investigation cost
- For developers:
 - identification of brittle feature dependencies
 - targeted model refinement
- For researchers:
 - principled causal analysis of IDS models
 - comparison of internal mechanisms across attacks

- This work extends mechanistic interpretability beyond NLP
- Introduces circuit-based explanations for NIDS
- Bridges security analytics and causal interpretability
- Enables:
 - explainable intrusion detection
 - auditable security decisions
 - robust and debuggable models

Limitations of ACDC for NIDS

- ACDC is computationally intensive
- Circuit discovery is performed offline
- Results depend on:
 - choice of corrupted traffic
 - behavior definition granularity
- Extracted circuits are:
 - model-specific
 - dataset-dependent

Threat Model and Adversarial Considerations

- Attackers may attempt to:
 - exploit spurious correlations in NIDS
 - evade detection via adversarial traffic
- ACDC enables:
 - identification of brittle internal circuits
 - analysis of attack-specific vulnerabilities
- This supports proactive hardening of NIDS models

Deployment Perspective in SOC Environments

- ACDC is not part of the real-time detection path
- It operates as:
 - an offline analysis and auditing module
- Outputs support:
 - analyst trust
 - incident post-mortem analysis
 - model debugging and retraining
- Compatible with existing SOC pipelines

Future Research Directions

- Scaling ACDC to:
 - deeper NIDS architectures
 - multi-attack joint circuits
- Integrating gradient-based pruning for efficiency
- Studying circuit stability across datasets
- Extending mechanistic interpretability to:
 - adaptive and evolving attacks

Conclusion

- Network intrusion detection requires more than accuracy
- ACDC enables:
 - causal understanding of NIDS decisions
 - attack-specific internal explanations
- This work bridges:
 - deep learning security
 - mechanistic interpretability
- Provides a foundation for trustworthy NIDS systems

Thank You!